Cake & Dessert Management System

Web Programming II — Laravel Coursework | Documentation draft

Creators: [Student 1 name and Neptun] / [Student 2 name and Neptun]

GitHub repository URL: [Add public project URL after publishing]

Live website URL: [Add URL after deployment]

Administrative login username: [Add admin user email]

FTP and database connection details: [Provide securely according to lecturer instructions; never upload credentials to public GitHub.]

IMPORTANT: This is a DRAFT, not final submission evidence. The project has not been deployed or captured in real browser screenshots. Replace every screenshot placeholder with a genuine screenshot and document real teamwork/deployment before submitting.

The homework mandates two-person collaboration, authentic GitHub activity, a running public website and a PDF describing screenshots and implementation.

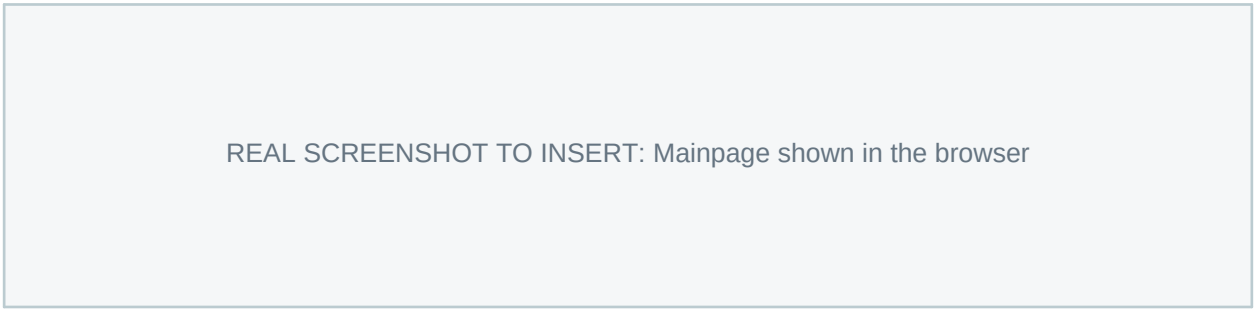
1. Project Overview & Objective

Purpose: present a confectionery catalogue using the three tables supplied by the lecturer: confections, contents and prices. The application also provides visitor messaging, statistics, access control, and administration.

Chosen theme: Bootswatch Minty, a free Bootstrap 5 responsive theme, with custom layout enhancements in public/css/site.css.

Technical stack: Laravel 12; PHP 8.2+; MySQL; Eloquent ORM; Blade templates; Bootstrap 5; Chart.js; GitHub for version control.

Public menu: Mainpage, Database, Contact, Graph, Login, Register. Registered users see Messages. Administrators additionally see CRUD Admin. Logout is available after login.



Implementation files: routes/web.php, resources/views/layouts/app.blade.php, resources/views/home.blade.php, app/Http/Controllers/HomeController.php.

2. Database Design & Relationships

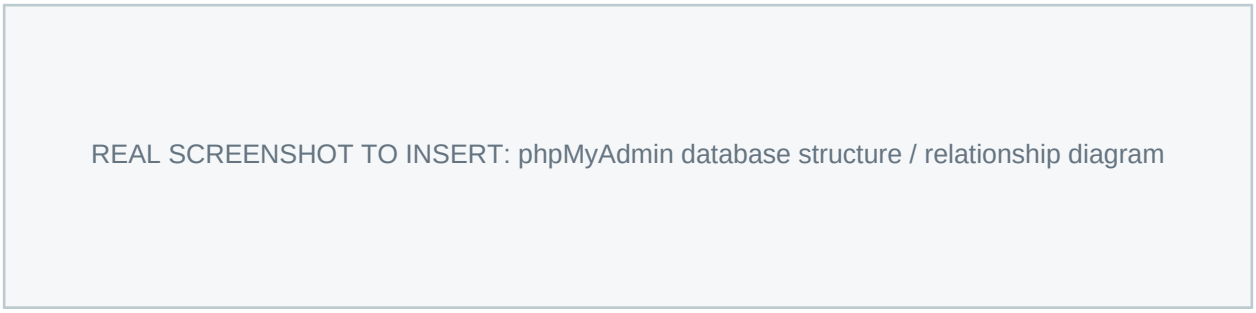
confections(id, cname, type, prizewinning): main catalogue. Source contains 139 rows.

contents(id, confid, free): diet-related codes associated with confections. Source contains 45 rows.

prices(id, confid, price, unit): prices associated with confections. Source contains 208 rows.

Relationships: confections.id -> contents.confid (one-to-many); confections.id -> prices.confid (one-to-many). Eloquent relationship functions are declared on Confection, Price and Content models.

The source includes original prizewinning values 0 and -1 even though the written description calls the field Boolean; the original values are preserved. The only explicitly decoded dietary code in the source is G = gluten-free.



Implementation files: app/Models/*.php; database/migrations; database/data/*.tsv; database/seeders/DatabaseSeeder.php.

3. Migration & Seeding

The application uses Laravel migrations to create users, confections, contents, prices and contact_messages tables. Foreign keys connect prices and contents to confections.

DatabaseSeeder loads the supplied tab-separated dataset. All original IDs, prices, units and free-from codes are retained. The source is stored under database/data so the project can be rebuilt without manual phpMyAdmin imports.

On a clean LOCAL test database, run: php artisan migrate:fresh --seed. This deletes existing tables; never run on a production database with important data.

To seed an admin user, set ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD in .env first. The admin password must be at least 12 characters.

REAL SCREENSHOT TO INSERT: Successful migration/seeding terminal output and row counts

4. Authentication & Roles

Authentication: registration, login and logout implemented with Laravel Auth sessions. Registration validates all data, hashes passwords automatically via the User model cast, and creates an ordinary registered user.

Visitor role: public pages only. Registered role: Messages menu in addition to public pages. Admin role: CRUD admin menu in addition to public pages.

The admin middleware uses a server-side role check. Laravel auth middleware restricts messages. Sessions are regenerated on login and invalidated on logout. CSRF tokens are included in all POST forms.

IMPORTANT COURSE DIFFERENCE: the lecturer requires extending the authentication solution from Solutions.zip. That ZIP was not provided. This project implements equivalent Laravel session authentication independently; merge with the lecturer starter to meet the exact course requirement.

REAL SCREENSHOT TO INSERT: Login / Register page and role-aware navigation

5. Database Menu and Eloquent ORM

The Database menu reads from all three original tables using Eloquent relationships and eager loading. It displays confection name/type/prizewinning, price options and units, and content codes.

Visitors can search by product name and filter by confection type. Results are paginated with 12 records per page.

Implementation: HomeController::database() uses Confection::with([prices, contents]). Blade renders the table in resources/views/database.blade.php.

Prices may be recorded per piece, kilogram, bar, wreath, or cake size (8/12/16/24 slices); these units are displayed without incorrect comparisons.

REAL SCREENSHOT TO INSERT: Database page, showing data from three source tables

6. Contact Form and Validation

The Contact menu presents name, email, subject and message fields. Validation is done on the server for required values, email format and string length. Invalid input returns errors; valid input is saved to contact_messages.

Security: Laravel CSRF protection, framework validation, escaped Blade output, rate limiting on submissions.

Implementation: resources/views/contact.blade.php; ContactController::create() and ::store(); routes/web.php; contact_messages migration and ContactMessage model.

REAL SCREENSHOT TO INSERT: Contact form with invalid submission and validation errors

Add a second screenshot of a successful submission and database record in the final report.

7. Messages and Access Control

A visitor who has not logged in cannot view /messages. Logged-in users can view submitted visitor messages, newest first, with the date and time of each message.

Implementation: ContactController::index() uses ContactMessage::latest()->paginate(10). The messages view displays sender, subject, message and created_at timestamp.

This requirement gives registered visitors access to messages and admin users access to the restricted management menu.

REAL SCREENSHOT TO INSERT: Messages page after successful login with timestamps

For your final evidence, also show the visitor redirect to the login page when attempting /messages without being authenticated.

8. Chart.js Graph

The Graph menu displays two charts generated from stored MySQL data. A bar chart shows the number of products of each confection type. A doughnut chart shows the number of price entries per unit.

HomeController::graph() aggregates results through Eloquent queries using COUNT and GROUP BY. Values are sent to the Blade view and rendered using Chart.js.

Implementation: resources/views/graph.blade.php; HomeController::graph(). Chart.js is loaded from the public CDN.

REAL SCREENSHOT TO INSERT: Graph page with live bar and doughnut charts

9. CRUD Administration & Tests

Administrators have Create, Read, Update and Delete actions for the confections table at /admin/confections. Other registered users receive HTTP 403 if they try to access the route.

Implementation: Admin/ConfectionController.php; resources/views/admin/*.blade.php; EnsureAdmin middleware; resource routes. Deletes cascade to the linked source price/content rows.

Automated test examples are included under tests/Feature/WebsiteTest.php. Run php artisan test with the needed PHP extensions and Composer packages installed. Record real test results; tests have not been run in the packaged environment.

REAL SCREENSHOT TO INSERT: CRUD table, edit form and command-line test results

10. GitHub, Hosting & Submission Evidence

Required: public GitHub repository, at least five distinct meaningful uploads/commits, visible work from both collaborators, and the names of contributors in the footer of every page.

Required: deploy on a hosting service where the application is actually running. Configure DB credentials and application key securely; use HTTPS; expose public/ as document root; set APP_DEBUG=false.

Record each contributor’s commits and affected files in docs/TEAM_TASKS.md. Do not fabricate contribution history. Replace all placeholder screenshot areas with real images before submission.

Final PDF filename must follow Name-NeptunCode.pdf. Each student submits the documentation PDF to Teams. The professor requires FTP, database, and admin login details in the final documentation; transmit sensitive credentials only via the lecturer-approved private submission channel, not a public repository.

REAL SCREENSHOT TO INSERT: GitHub commit history and public hosted website

Final checklist: real screenshots; personal identifiers; functioning hosting; private credential delivery; source contribution evidence; completion of original authentication starter requirement.